Progress and Preservation

Concepts of Programming Languages

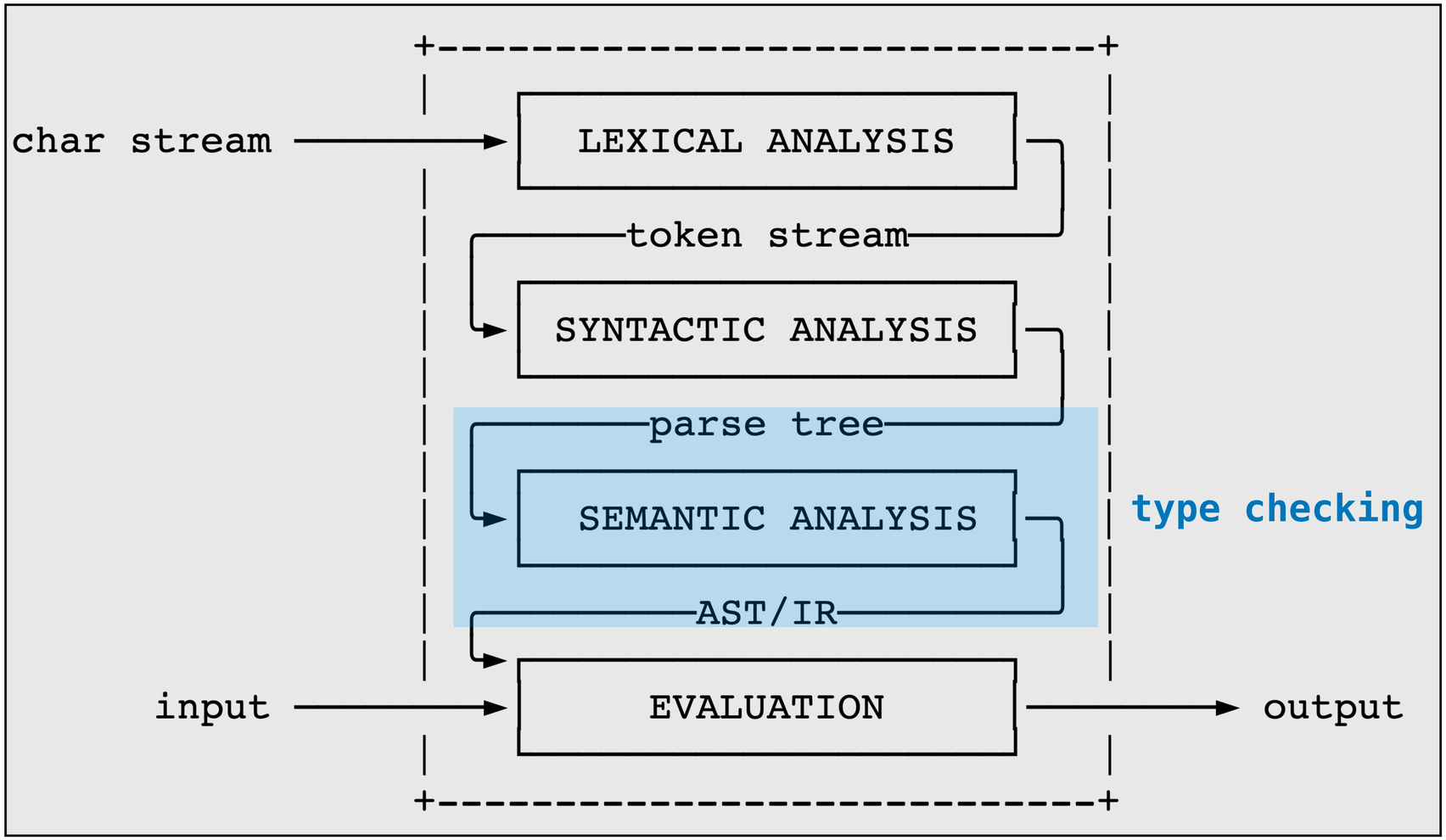
CAS CS 320

Outline

- Finish demo of STLC
- Discuss **induction** over derivations
- Show that STLC satisfies **progress** and **preservation**

Recap

Recall: The Picture



Recall: Type Checking/Inference

```
type_check : expr -> ty -> bool

type_of : expr -> ty option
```

Recall: Type Checking/Inference

```
type_check : expr -> ty -> bool

type_of : expr -> ty option
```

Type checking: determining whether an expression can be typed

Recall: Type Checking/Inference

```
type_check : expr -> ty -> bool

type_of : expr -> ty option
```

Type checking: determining whether an expression can be typed

Type inference: *synthesizing* a type for an expression

Recall: Type Checking/Inference

```
type_check : expr -> ty -> bool

type_of : expr -> ty option
```

Type checking: determining whether an expression can be typed

Type inference: *synthesizing* a type for an expression

Theoretically, these two problems can be very different. *For STLC, they are both easy*

Recall: Syntax (STLC)

$$e ::= x \mid \lambda x^{\tau}.e \mid ee$$

$$\tau ::= \bot \mid \tau \to \tau$$

Recall: Typing (STLC)

$$\frac{(x : \tau) \in \Gamma}{\Gamma \vdash x : \tau}\ \textsf{var} \qquad \frac{\Gamma, x : \tau \vdash e : \tau'}{\Gamma \vdash \lambda x^{\tau} . e : \tau \to \tau'}\ \textsf{fun}$$

$$\frac{\Gamma \vdash e_1 : \tau \to \tau' \qquad \Gamma \vdash e_2 : \tau}{\Gamma \vdash e_1 e_2 : \tau'}\ \textsf{app}$$

Recall: Semantics (STLC)

$$\frac{e_1 \longrightarrow e_1'}{e_1 e_2 \longrightarrow e_1' e_2}\ \texttt{leftApp}$$

$$\frac{}{(\lambda x\,.\,e)e' \longrightarrow [e'/x]e}\ \texttt{beta}$$

Recall: Semantics (STLC)

$$\frac{e_1 \longrightarrow e_1'}{e_1 e_2 \longrightarrow e_1' e_2}\ \texttt{leftApp}$$

$$\frac{}{(\lambda x\,.\,e)e' \longrightarrow [e'/x]e}\ \texttt{beta}$$

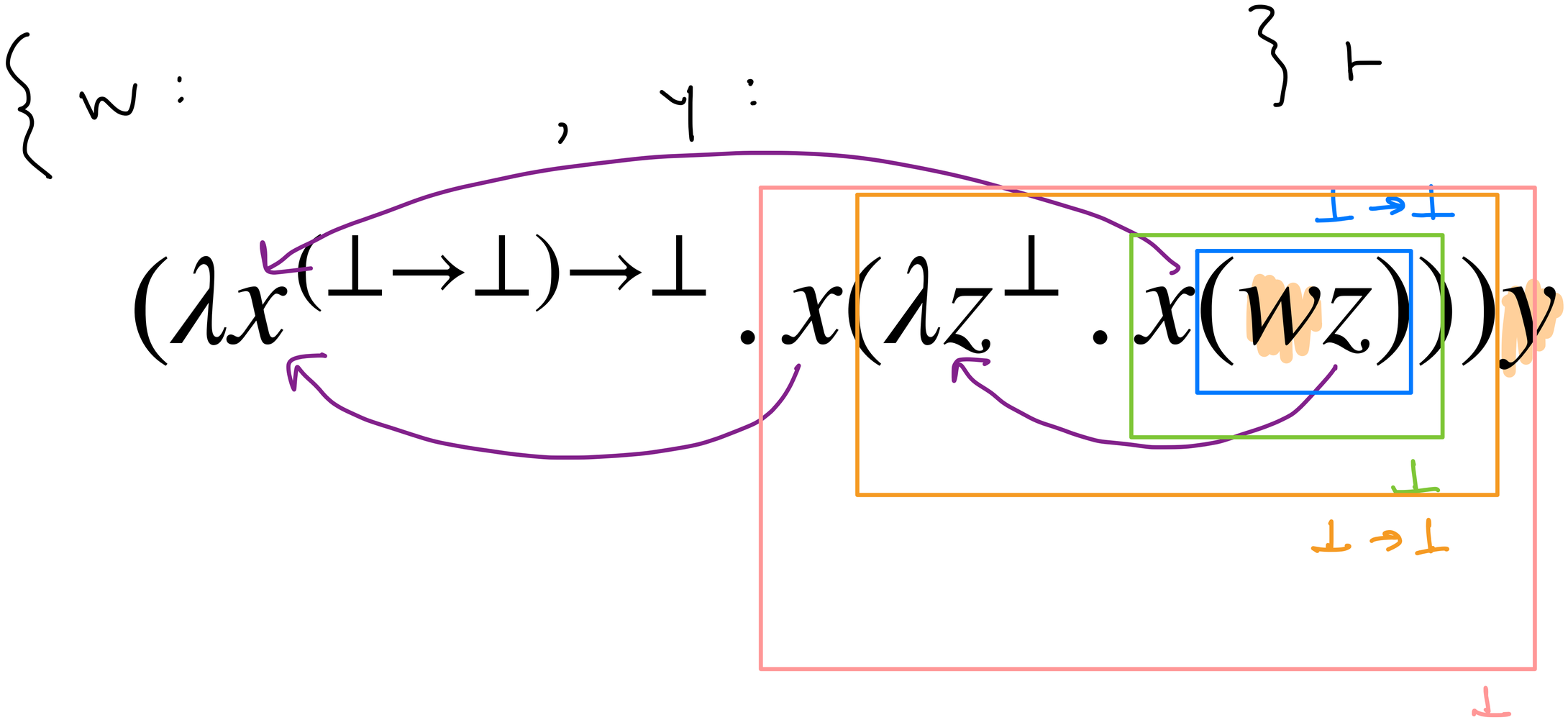
We can use any semantics (this is small-step CBN)

Practice Problem

$$(\lambda x^{(\bot \to \bot) \to \bot} . x(\lambda z^{\bot} . x(wz)))y$$

Determine the smallest context such that the above expression is well-typed (also give its type)

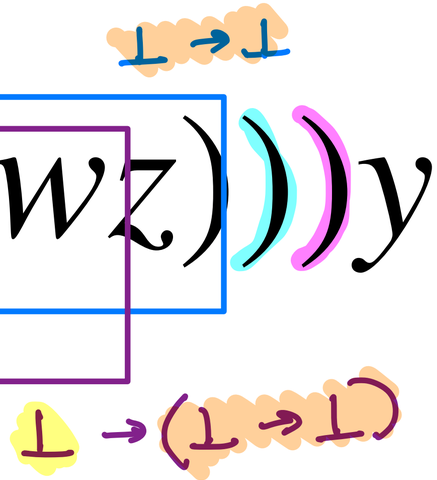
Answer



Answer

$\{ w : \bot \to (\bot \to \bot),\ y : (\bot \to \bot) \to \bot \} \vdash$

$$(\lambda x^{(\bot \to \bot) \to \bot} . x(\lambda z^{\bot} . x(wz)))y : \bot$$

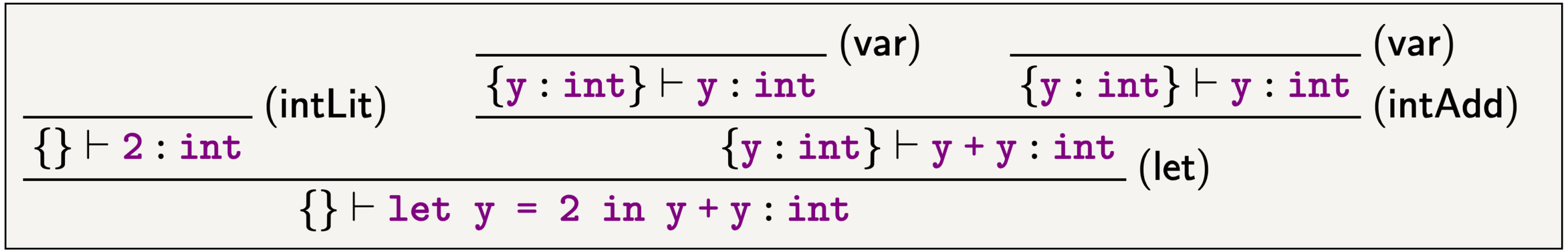


demo

Induction over Derivations

The Key Idea

The Key Idea



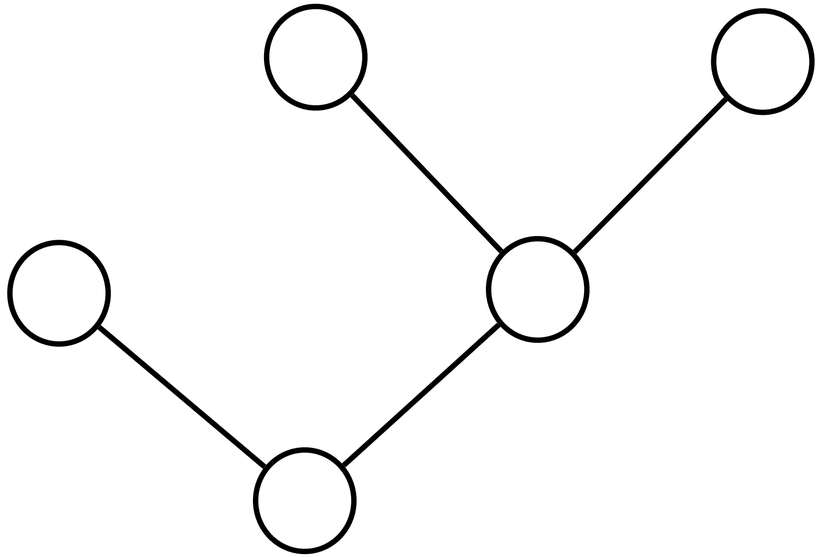
Derivations are *trees*

The Key Idea

$$\dfrac{\dfrac{}{\{\} \vdash \texttt{2} : \texttt{int}}\,(\text{intLit}) \qquad \dfrac{\dfrac{}{\{\texttt{y} : \texttt{int}\} \vdash \texttt{y} : \texttt{int}}\,(\text{var}) \qquad \dfrac{}{\{\texttt{y} : \texttt{int}\} \vdash \texttt{y} : \texttt{int}}\,(\text{var})}{\{\texttt{y} : \texttt{int}\} \vdash \texttt{y} + \texttt{y} : \texttt{int}}\,(\text{intAdd})}{\{\} \vdash \texttt{let y = 2 in y + y} : \texttt{int}}\,(\text{let})$$

Derivations are *trees*

We can prove things about trees using induction



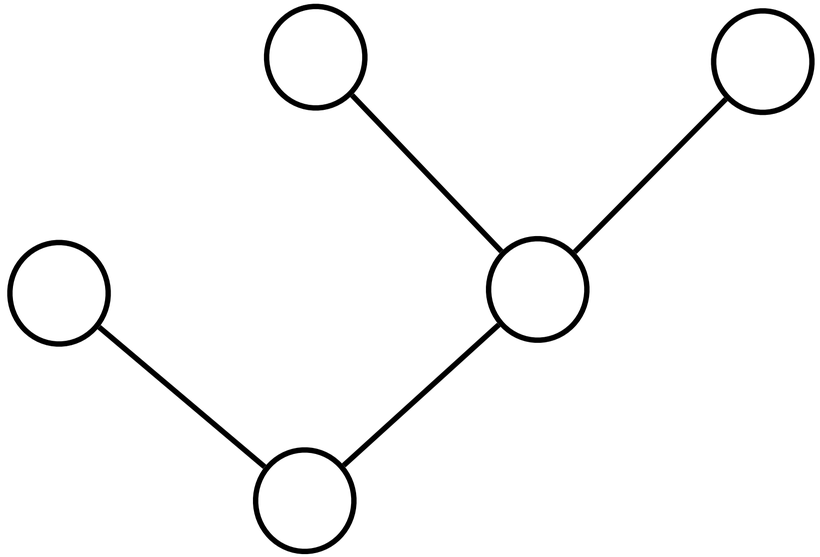
The Key Idea

$$\dfrac{\dfrac{}{\{\} \vdash \texttt{2} : \texttt{int}}\,(\text{intLit}) \qquad \dfrac{\dfrac{}{\{\texttt{y} : \texttt{int}\} \vdash \texttt{y} : \texttt{int}}\,(\text{var}) \qquad \dfrac{}{\{\texttt{y} : \texttt{int}\} \vdash \texttt{y} : \texttt{int}}\,(\text{var})}{\{\texttt{y} : \texttt{int}\} \vdash \texttt{y} + \texttt{y} : \texttt{int}}\,(\text{intAdd})}{\{\} \vdash \texttt{let y = 2 in y + y} : \texttt{int}}\,(\text{let})$$

Derivations are *trees*

We can prove things about trees using induction

We can prove things about *derivable judgments* using induction



The Key Idea

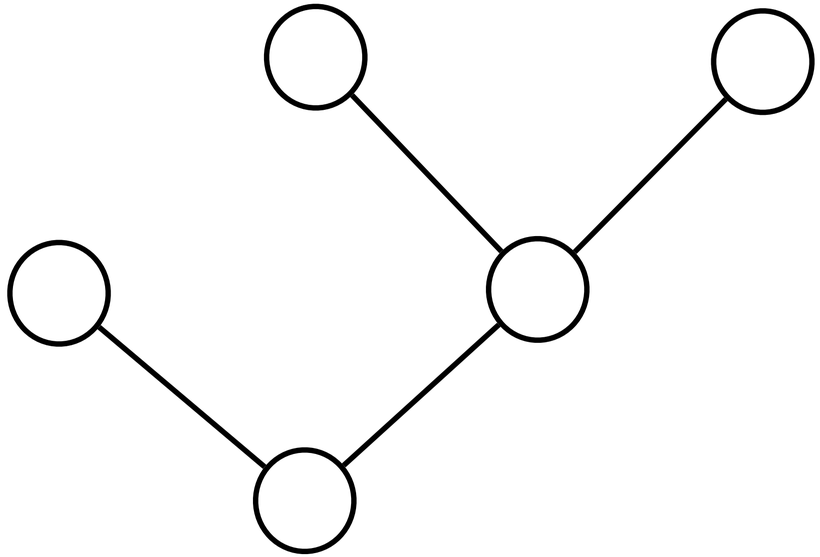
$$\dfrac{\dfrac{}{\{\} \vdash \texttt{2} : \texttt{int}}\,(\text{intLit}) \qquad \dfrac{\dfrac{}{\{\texttt{y} : \texttt{int}\} \vdash \texttt{y} : \texttt{int}}\,(\text{var}) \qquad \dfrac{}{\{\texttt{y} : \texttt{int}\} \vdash \texttt{y} : \texttt{int}}\,(\text{var})}{\{\texttt{y} : \texttt{int}\} \vdash \texttt{y} + \texttt{y} : \texttt{int}}\,(\text{intAdd})}{\{\} \vdash \texttt{let y = 2 in y + y} : \texttt{int}}\,(\text{let})$$

Derivations are *trees*

We can prove things about trees using induction

We can prove things about *derivable judgments* using induction

Important: Every derivable judgment corresponds to a derivation



Induction over $\mathbb{N}$



Given property P on $\mathbb{N}$: If

(i) $P(0)$ (base case)

(ii) $\forall k.\ \underbrace{P(k)}_{\text{induction hypothesis}} \Rightarrow P(k+1)$ (induction step)

Then $\forall n.\ P(n)$ P holds of all nat. nums.

Induction over Derivations

Given property P over judgements, if

ex. $\dfrac{}{\Gamma \vdash x : \tau}$ (var) $\Rightarrow P(\Gamma \vdash x : \tau)$ holds

(1) P holds of axioms

(2) Given rule of the form

$$\frac{J_1 \quad \ldots \quad J_k}{J}$$

$P(J_1), \ldots, P(J_k) \Rightarrow P(J)$

Then P holds of all judgements

Warm-up: Well-Scopedness

Def. e is well-scoped w.r.t. Γ if $x \in FV(e)$ implies there is τ, $(x:\tau) \in \Gamma$

Prop. $\Gamma \vdash e : \tau$ (with $\mathcal{D}$) then e is well-scoped w.r.t. Γ.

Proof By induction on $\mathcal{D}$.

Warm-up: Well-Scopedness

Warm-up: Well-Scopedness

An expression e is **well-scoped** with respect to a context Γ if $x \in FV(e)$ implies x appears in Γ

Warm-up: Well-Scopedness

An expression e is **well-scoped** with respect to a context Γ if $x \in FV(e)$ implies x appears in Γ

> **Theorem.** If e is well-typed in Γ, then e is well-scoped

Warm-up: Well-Scopedness

An expression e is **well-scoped** with respect to a context Γ if $x \in FV(e)$ implies x appears in Γ

> **Theorem.** If e is well-typed in Γ, then e is well-scoped

Proof. By induction on *derivations*

Base Case: Axioms

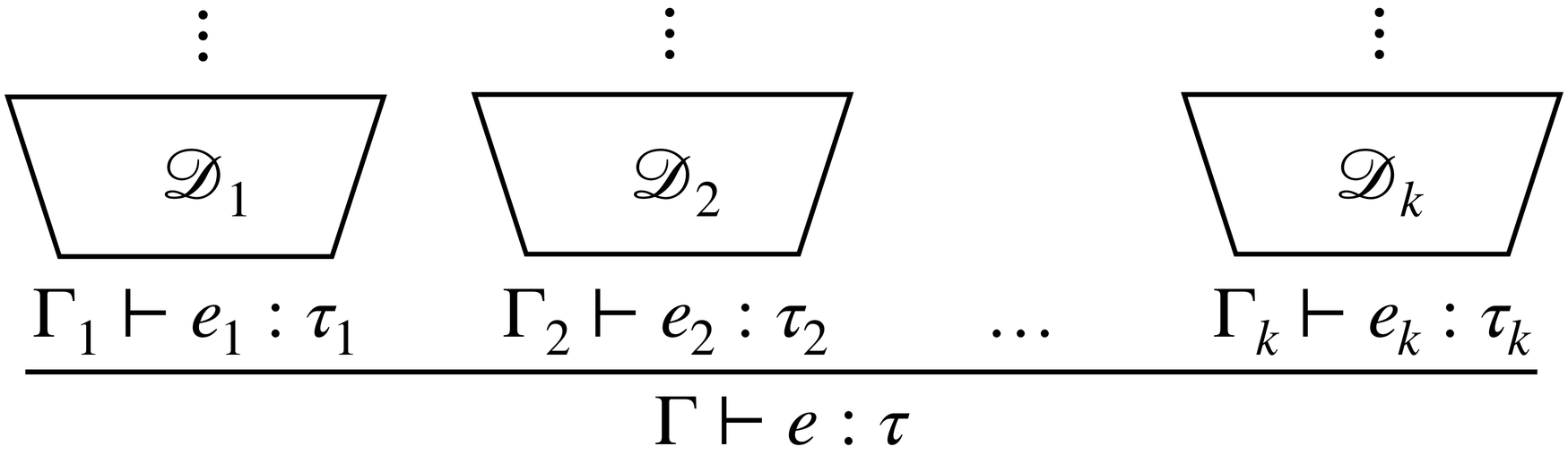
$$\frac{x : T \in \Gamma}{\Gamma \vdash x : T}\ (\text{var})$$

immediately follows from side cond.

WTS. $e = x$ is well scoped

$$FV(e) = FV(x) = \{x\}$$

Inductive Hypothesis



$e_1, \ldots, e_k$ *are well-scoped (because they're well-typed in the each of their contexts)*

Inductive Step 1: Application

$$\frac{\Gamma \vdash e_1 : T_2 \to T \quad (D_1) \qquad \Gamma \vdash e_2 : T_2 \quad (D_2)}{\Gamma \vdash e_1\, e_2 : T}$$

WTS. $e_1 e_2$ is well-scoped w.r.t. Γ

$x \in FV(e_1 e_2) = FV(e_1) \cup FV(e_2)$

$\Downarrow$

$x \in FV(e_1)$ or $x \in FV(e_2)$

(i) $x \in FV(e_1)$

e_1 is well-typed by D_1

so e_1 is well-scoped by IH

$\Downarrow$

x is declared in Γ.

(ii) $x \in FV(e_2)$

(similar)

Inductive Step 2: Function

$$\frac{\Gamma, x : \tau_1 \vdash e : \tau_2}{\Gamma \vdash \lambda x^{\tau_1}.e : \tau_1 \to \tau_2}\ (\text{fun})$$

D'

WTS: $\lambda x^{\tau_1}.e$ is well-scoped

$y \in FV(\lambda x^{\tau}.e) = FV(e) \setminus \{x\}$

$\Downarrow$

$y \in FV(e)$ and $y \neq x$

e is well-typed by D'

so by IH e is well-scoped

$\Downarrow$

y is declared in $\Gamma, x : \tau_1$

$y \neq x \Rightarrow$ y is declared in Γ.

What if the last rule I applied was abstraction?

Progress and Preservation

Recall: Type Safety

Recall: Type Safety

> **Thm.** If $\cdot \vdash e : \tau$ and there is a value v such that $\langle \varnothing, e \rangle \Downarrow v$ then $\cdot \vdash v : \tau$

Recall: Type Safety

Thm. If $\cdot \vdash e : \tau$ and there is a value v such that $\langle \varnothing, e \rangle \Downarrow v$ then $\cdot \vdash v : \tau$

With small-step semantics, we can give a finer-grained analysis:

Recall: Type Safety

Thm. If $\cdot \vdash e : \tau$ and there is a value v such that $\langle \varnothing, e \rangle \Downarrow v$ then $\cdot \vdash v : \tau$

With small-step semantics, we can give a finer-grained analysis:

Thm.

» *(progress)* If $\cdot \vdash e : \tau$ then e is a value or $e \longrightarrow e'$ for some e'

» *(preservation)* If $\Gamma \vdash e : \tau$ and $e \longrightarrow e'$ then $\Gamma \vdash e' : \tau$

Recall: Type Safety

> **Thm.** If $\cdot \vdash e : \tau$ and there is a value v such that $\langle \varnothing, e \rangle \Downarrow v$ then $\cdot \vdash v : \tau$

With small-step semantics, we can give a finer-grained analysis:

> **Thm.**
>
> » *(progress)* If $\cdot \vdash e : \tau$ then e is a value or $e \longrightarrow e'$ for some e'
>
> » *(preservation)* If $\Gamma \vdash e : \tau$ and $e \longrightarrow e'$ then $\Gamma \vdash e' : \tau$

These results are *fundamental*. They tell us that our PL is well-behaved (it's a "good" PL)

Recall: Type Safety

> **Thm.** If $\cdot \vdash e : \tau$ and there is a value v such that $\langle \varnothing, e \rangle \Downarrow v$ then $\cdot \vdash v : \tau$

With small-step semantics, we can give a finer-grained analysis:

goal for today

> **Thm.**
>
> » *(progress)* If $\cdot \vdash e : \tau$ then e is a value or $e \longrightarrow e'$ for some e'
>
> » *(preservation)* If $\Gamma \vdash e : \tau$ and $e \longrightarrow e'$ then $\Gamma \vdash e' : \tau$

These results are *fundamental*. They tell us that our PL is well-behaved (it's a "good" PL)

Recall: STLC

Syntax

$$e ::= x \mid \lambda x^{\tau}.\, e \mid ee$$

$$\tau ::= \bot \mid \tau \to \tau$$

Typing

$$\frac{(x : \tau) \in \Gamma}{\Gamma \vdash x : \tau}\ \textbf{var} \qquad \frac{\Gamma, x : \tau \vdash e : \tau'}{\Gamma \vdash \lambda x^{\tau}.\, e : \tau \to \tau'}\ \textbf{fun}$$

$$\frac{\Gamma \vdash e_1 : \tau \to \tau' \qquad \Gamma \vdash e_2 : \tau}{\Gamma \vdash e_1 e_2 : \tau'}\ \textbf{app}$$

Semantics

$$\frac{e_1 \longrightarrow e_1'}{e_1 e_2 \longrightarrow e_1' e_2}\ \textbf{leftApp} \qquad \frac{}{(\lambda x.\, e) e' \longrightarrow [e'/x]e}\ \textbf{beta}$$

Progress (STLC)

> **Theorem.** If e is well-typed then e is a value or there is an expression e' such that $e \longrightarrow e'$

Proof. By induction over *derivations*



Base Case: Variables

$$\frac{(x : \tau) \in \emptyset}{\emptyset \vdash x : \tau}\ (\text{var})$$

can never make true because $\Gamma = \emptyset$

IMPOSSIBLE. NOTHING TO SHOW

Inductive Step 1: Application

$$\frac{\emptyset \vdash e_1 : \tau_2 \to \tau \qquad \emptyset \vdash e_2 : \tau_2}{\emptyset \vdash e_1\, e_2 : \tau}$$

By IH on $\mathcal{D}_1$

① e_1 is value $(\lambda x{:}\tau.\, e')$

② $\exists e_1'.\ e_1 \to e_1'$

W.T.S. ① $e_1\, e_2$ is value ✗ not possible, values are funcs.

② $\exists e'$ s.t. $e_1\, e_2 \to e'$

e_1 depends on e_1

① $e_1 e_2 = (\lambda x^{\tau}.\, e')\, e_2 \to [e_2/x]e'$ (by β)

② $\dfrac{e_1 \to e_1'}{e_1\, e_2 \longrightarrow e_1'\, e_2}$ (by leftapp)

Inductive Step 2: Functions

$$\frac{\{x:\tau'\} \vdash e' : \tau''}{\emptyset \vdash \lambda x^{\tau'}.e' : \tau' \to \tau''} \text{(fun)}$$

already a value

Preservation (STLC)

Theorem. If $\Gamma \vdash e : \tau$ and $e \longrightarrow e'$ then $\Gamma \vdash e' : \tau$

Proof. By induction over *derivations*

This one is tricker...

Base Case: Vars

Inductive Step 1: Functions

Inductive Step 2: Application

Inductive Step 2.1: leftEval

Inductive Step 2.1: Beta

Substitution Lemma

Lemma. If $\Gamma \vdash e_2 : \tau_2$ and $\Gamma, x : \tau_2 \vdash e : \tau$ then

$$\Gamma \vdash [e_2/x]e : \tau$$

That is, if e is well-typed in a context with (x, τ) then we can substitute x with anything of type τ and it's still the same type

(we can prove this by, you guessed it, induction on derivations)

The Point

```
let rec eval env e =
  match e with
  | Var x -> Env.find x env
  ...
```

Progress and preservation tell us that **terms never get stuck during evaluation**

Our type system ensures we only evaluate programs that make sense.

Summary

Progress and **preservation** are fundamental features of good programming languages

We can prove things about well-typed expressions by performing **induction** over derivations